

Festival

Nayra está em um festival jogando um jogo cujo grande prêmio é uma viagem para Laguna Colorada. O jogo consiste em usar fichas para comprar cupons. Comprar um cupom pode resultar em fichas adicionais. O objetivo é conseguir o máximo de cupons possível.

Ela começa o jogo com A fichas. Existem N cupons, numerados de 0 a $N - 1$. Nayra tem que pagar $P[i]$ fichas ($0 \leq i < N$) para comprar o cupom i (e ela deve ter pelo menos $P[i]$ fichas antes da compra). Ela pode comprar cada cupom no máximo uma vez.

Além disso, cada cupom i ($0 \leq i < N$) possui um **tipo**, denotado por $T[i]$, que é um número inteiro **entre 1 e 4, inclusive**. Depois que Nayra compra o cupom i , o número de fichas restantes que ela possui é multiplicado por $T[i]$. Formalmente, se ela tiver X fichas em algum momento do jogo, e compra o cupom i (que requer $X \geq P[i]$), então ela terá $(X - P[i]) \cdot T[i]$ fichas após a compra.

Sua tarefa é determinar quais cupons Nayra deve comprar e em que ordem, para maximizar o número total de **cupons** que ela tem no final. Se houver mais de uma sequência de compras que consiga isso, você pode relatar qualquer uma delas.

Detalhes de implementação

Você deve implementar o seguinte procedimento.

```
std::vector<int> max_coupons(int A, std::vector<int> P,  
                             std::vector<int> T)
```

- A : o número inicial de fichas que Nayra possui.
- P : um vetor de comprimento N especificando os preços dos cupons.
- T : um vetor de comprimento N especificando os tipos dos cupons.
- Este procedimento é chamado exatamente uma vez para cada caso de teste.

O procedimento deve retornar um vetor R , que especifica as compras de Nayra da seguinte forma:

- O comprimento de R deve ser o número máximo de cupons que ela pode comprar.
- Os elementos do vetor são os índices dos cupons que ela deve comprar, em ordem cronológica. Ou seja, ela compra o cupom $R[0]$ primeiro, o cupom $R[1]$ em segundo lugar, e

assim por diante.

- Todos os elementos de R devem ser únicos.

Se nenhum cupom puder ser comprado, R deverá ser um vetor vazio.

Restrições

- $1 \leq N \leq 200\,000$
- $1 \leq A \leq 10^9$
- $1 \leq P[i] \leq 10^9$ para cada i tal que $0 \leq i < N$.
- $1 \leq T[i] \leq 4$ para cada i tal que $0 \leq i < N$.

Subtarefas

Subtarefa	Pontuação	Restrições adicionais
1	5	$T[i] = 1$ para cada i tal que $0 \leq i < N$.
2	7	$N \leq 3000$; $T[i] \leq 2$ para cada i tal que $0 \leq i < N$.
3	12	$T[i] \leq 2$ para cada i tal que $0 \leq i < N$.
4	15	$N \leq 70$
5	27	Nayra pode comprar todos os N cupons (em alguma ordem).
6	16	$(A - P[i]) \cdot T[i] < A$ para cada i tal que $0 \leq i < N$.
7	18	Sem restrições adicionais.

Exemplos

Exemplo 1

Considere a seguinte chamada.

```
max_coupons(13, [4, 500, 8, 14], [1, 3, 3, 4])
```

Nayra tem inicialmente $A = 13$ fichas. Ela pode comprar 3 cupons na ordem mostrada abaixo:

Cupom comprado	Preço do cupom	Tipo de cupom	Número de fichas após a compra
2	8	3	$(13 - 8) \cdot 3 = 15$
3	14	4	$(15 - 14) \cdot 4 = 4$
0	4	1	$(4 - 4) \cdot 1 = 0$

Neste exemplo, não é possível para Nayra comprar mais de 3 cupons, e a sequência de compras descrita acima é a única maneira de ela conseguir comprar 3 deles. Portanto, o procedimento deve retornar $[2, 3, 0]$.

Exemplo 2

Considere a seguinte chamada.

```
max_coupons(9, [6, 5], [2, 3])
```

Neste exemplo, Nayra pode comprar os dois cupons em qualquer ordem. Portanto, o procedimento deve retornar $[0, 1]$ ou $[1, 0]$.

Exemplo 3

Considere a seguinte chamada.

```
max_coupons(1, [2, 5, 7], [4, 3, 1])
```

Neste exemplo, Nayra tem uma ficha, que não é suficiente para comprar nenhum cupom. Portanto, o procedimento deve retornar $[]$ (um vetor vazio).

Corretor de Exemplo

Formato de entrada:

```
N A
P[0] T[0]
P[1] T[1]
...
P[N-1] T[N-1]
```

Formato de saída:

```
S
R[0] R[1] ... R[S-1]
```

Aqui, S é o comprimento do vetor R retornado por `max_coupons`.